Database Schema and Architecture

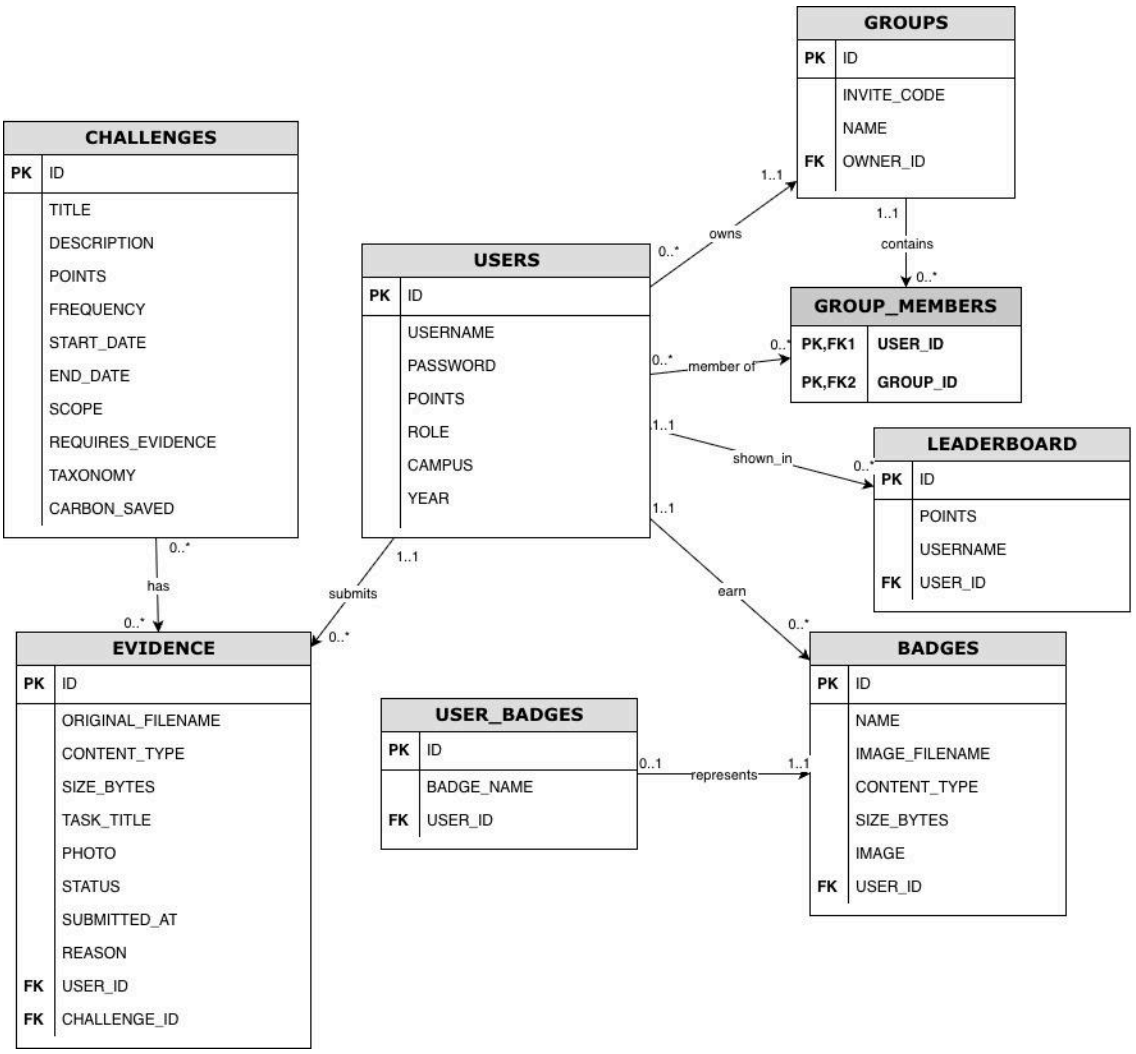
Introduction:

The database captures all the essential information required to support the Footprint game, including tracking user activity and badges while also maintaining leaderboards and information about challenges. It is implemented using PostgreSQL with the mappings being managed through the Jakarta Persistence API (JPA).

The database has been created with certain goals in mind. Firstly it has been designed in a user-centric manner, where badges, evidence submissions and points are all connected to users. This is done to support reporting and to ensure that the user is the central entity in the system. Another goal the database has been designed with is data consistency and integrity whereby the primary keys, foreign keys and constraints force uniqueness, ensuring that the data is properly linked together and is valid.

Entity-Relationship Diagram (ERD) and Relationships:

The database is structured around some core entities that reflect the main workflow of the game process. The following UML-style ER diagram captures the relational database visually:



The ER Diagram reflects a user-centric design, tracking each individual's activity on the game. All entities are directly or indirectly connected to the User, allowing for the benefits that User-centric design provides, such as helping with accurate analytics and reporting.

Users and Groups:

Each group has a single owner, and the two entities are linked through the OWNER_ID foreign key, in GROUPS, which references ID in the USERS entity. Meanwhile, users can belong to multiple groups, and there is a join table of GROUPS_MEMBERS, due to a many-to-many relationship. This difference between ownership of a group and membership of a group allows for efficient querying when calculating total group points or when listing the members of the group.

Users and Evidence:

Each evidence submission is associated with exactly one user so that there is proper accountability. Not all challenges require verification through evidence, so there is an optional reference between EVIDENCE and CHALLENGES to be able to handle both types

of challenges, while also maintaining data integrity. The PHOTO attribute itself is stored as a binary large object (BLOB) in order to be able to accommodate images and the SUBMITTED_AT attribute notes the timestamp of when a submission is made.

Challenges and Evidence:

A single challenge can be done by multiple users, therefore one challenge can have multiple submissions of evidence from different users. The one-to-many relationship enables for the system to track the overall completion of the challenges, and users' individual completions of the challenges. Separating Challenges and Evidence ensures a separate storage of game logic and user activity, so that each entity stores only relevant details.

Users and Badges:

Users can earn multiple badges, with each badge being linked to a single user. There is a USER_BADGES table tracking which badge is currently being displayed by a user. Separating of the earned badges earned, from the badges displayed, allows users to change the display, without affecting their record of earned badges. NAME and IMAGE attributes are constrained to ensure they are unique and consistent.

Users and Leaderboard Entries:

Leaderboard entries show snapshots of user rankings and points. Each entry is tied to a single user, so that the leaderboard can be rebuilt efficiently without needing to join multiple tables. It supports the quick retrieval of rankings and maintains the consistency of users' points.

Data Integrity and Constraints:

We attempt to maintain data integrity through constraints. Primary keys, foreign keys and uniqueness constraints ensure that we can enforce valid relationships and unique values. An example of this is that badges are uniquely identified by USER_ID and NAME, so that there are no duplicate achievements. In a similar way USER_BADGES, enforces a one-to-one relationship between a user and the badge that they are displaying. These constraints ensure that there is data consistency, also allowing the analytics displayed to the users to be reliable.

Data Dictionary

Definition of data fields & naming conventions

USERS table:

Column	Data Type	Constraints/Keys	Description
ID	BIGINT	PK	Unique identifier for user
USERNAME	VARCHAR	UNIQUE, NOT NULL	Login name, alphanumeric, minimum 3 characters
PASSWORD	VARCHAR	NOT NULL	Hashed password
POINTS	INT	DEFAULT 0	Total points accumulated by the user
ROLE	VARCHAR	NOT NULL	User role: USER or MODERATOR
CAMPUS	VARCHAR	DEFAULT ‘streatham’	User’s campus location
YEAR	VARCHAR	DEFAULT ‘First_Year’	Academic Year or Staff

GROUPS table

Column	Data Type	Constraints/Keys	Description
ID	INT	PK	Unique identifier for group
NAME	VARCHAR	NOT NULL	Group name
INVITE_CODE	VARCHAR	UNIQUE	Code for joining the group
OWNER_ID	BIGINT	FK: USERS.ID	User who owns the group

GROUP_MEMBERS table

Column	Data Type	Constraints/Keys	Description
USER_ID	BIGINT	PK, FK: USERS.ID	User who is a member of the group
GROUP_ID	INT	PK, FK: GROUPS.ID	Group the user belongs to

CHALLENGES table

Column	Data Type	Constraints/Keys	Description
ID	BIGINT	PK	Unique challenge ID
TITLE	VARCHAR	NOT NULL	Challenge name/title
DESCRIPTION	VARCHAR		Description of the challenge
POINTS	INT	NOT NULL	Points awarded for completing challenge
FREQUENCY	VARCHAR		How often the challenge appears
START_DATE	DATE		When challenge becomes available
END_DATE	DATE		When challenge ends
SCOPE	VARCHAR		Scope of the challenge
REQUIRES_EVIDENCE	BOOLEAN	NOT NULL	Whether the challenge requires evidence to be submitted
TAXONOMY	VARCHAR		Category of challenge
CARBON_SAVED	DOUBLE		Estimated CO2 reduced in kg

LEADERBOARD table

Column	Data Type	Constraints/Keys	Description
ID	BIGINT	PK	Unique leaderboard entry ID
USER_ID	BIGINT	FK: USERS.ID, NOT NULL	User associated with the leaderboard entry
USERNAME	VARCHAR	NOT NULL	Username for display

POINTS	INT	NOT NULL	Points for display for leaderboard ranking

BADGES table

Column	Data Type	Constraints/Keys	Description
ID	BIGINT	PK	Unique badge ID
USER_ID	BIGINT	FK: USERS.ID, NOT NULL	User who earned the badge
NAME	VARCHAR	NOT NULL, UNIQUE	Name of the badge
IMAGE_FILENAME	VARCHAR		File name of the badge image
CONTENT_TYPE	VARCHAR		Type of the badge image
SIZE_BYTES	BIGINT		File size of image in bytes
IMAGE	BLOB		Binary image data

USER_BADGES table

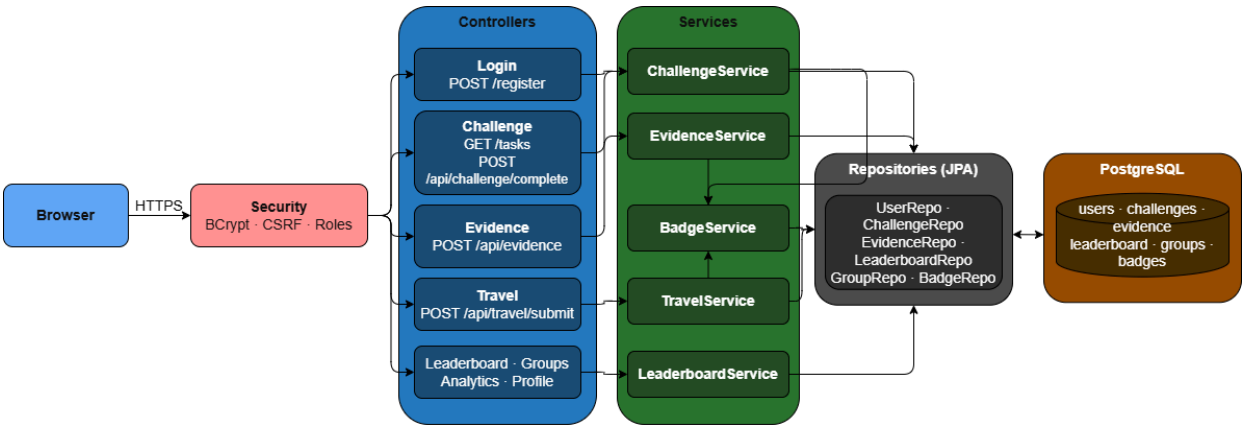
Column	Data Type	Constraints/Keys	Description
ID	BIGINT	PK	Unique ID
USER_ID	BIGINT	FK: USERS.ID, NOT NULL	User who selected the badge
BADGE_NAME	VARCHAR	NOT NULL	Name of the badge currently displayed

EVIDENCE table

Column	Data Type	Constraints/Keys	Description
ID	BIGINT	PK	Unique evidence ID for submission
USER_ID	BIGINT	FK: USERS.ID, NOT NULL	User who has submitted evidence
CHALLENGE_ID	BIGINT	FK:	Challenge that

		CHALLENGES.ID	evidence has been submitted for
ORIGINAL_FILENAME	VARCHAR	NOT NULL	Original file name of image
CONTENT_TYPE	VARCHAR	NOT NULL	Type of uploaded image
SIZE_BYTES	BIGINT		Size of image in bytes
TASK_TITLE	VARCHAR		Title of the task
PHOTO	BLOB	NOT NULL	Binary image data
STATUS	VARCHAR	NOT NULL	Moderation status
SUBMITTED_AT	TIMESTAMP	NOT NULL	Timestamp when evidence was submitted
REASON	VARCHAR		Moderator comment for approving or rejecting

Data Flow Diagram



The Data Flow diagram reflects the controller-service-repository pattern used in the backend. The diagram illustrates how user requests move through the different layers in our system, going from the browser, to the backend and ultimately to the database. While the diagram contains multiple controllers and service interactions, it generally follows a consistent layered structure.

The flow starts with the user interacting with the system through the browser, whereby requests are sent over HTTPS and pass through a security layer which is responsible for handling authentication and protection mechanisms. These include BCrypt password hashing, role-based access control and CSRF protection. This ensures that only users who are authorised for specific functionality, can access that functionality.

After passage through the security layer, requests are then routed to the relevant controllers. These controllers act as entry points, for different features of the system such as, evidence submission, travel logging, leaderboard, group management, challenges and authentication. Each controller's role is to receive a request and forward it onwards to the relevant service in the service layer, which will then process it.

As we mentioned, the request moves to the service layer which is where the main application logic lies and where most of the processing happens. Services like ChallengeService and EvidenceService handle tasks like verifying the user's challenge completion, awarding points and managing the relationships between different entities. The flow diagram shows how rather than being isolated, different parts of the system are interconnected, because some requests may pass through multiple services.

At a lower level, services interact with the classes in the repository layer to do database operations. There, the repositories use JPA in order to communicate with the PostgreSQL, so that data can be stored properly and retrieved efficiently, still ensuring that database operations and logic are separate from the logic which is for the main application. Overall, the diagram shows a structured flow of data through these clearly defined layers. Hence despite multiple interactions between components, the separation between these separate layers and what role they play, ensures that the system can maintain consistency, can be scaled easily and can be easily maintained in terms of how it handles data and processes data.

Backup and Recovery

Backing up the production database

Connecting to the database.

Install psql if you don't already have it (included with PostgreSQL), then connect using the values from the dashboard:

```
psql -h <external-host> -U <username> <database-name>
```

You will be prompted for the password.

Create a backup:

Use `pg_dump` to create a backup of the PostgreSQL database.

```
pg_dump -h <external-host> -U <username> -d <database-name> -F c -f
backup.dump
```

- `-F c` produces a custom-format archive (compressed, suitable for `pg_restore`).

- You will be prompted for the password, or you can set the `PGPASSWORD` environment variable to avoid the prompt:

```
PGPASSWORD=<password> pg_dump -h <external-host> -U <username> -d
<database-name> -F c -f backup.dump
```

Restore from a backup:

To restore data from the backup created, enter this command:

```
pg_restore -h <external-host> -U <username> -d <database-name> -F c
backup.dump
```

This will load all of the data that was backed up into the active tables, overriding existing data. To ensure you minimise data lost when restoring from backup ensure that backups of the database are regularly taken

The exact details used for our global host and what data to use in these commands can be found in the Handover Index, this can be found in `2_handover_pack/handover_index.pdf`.

Access and Security

Database Access

The production database is a PostgreSQL instance hosted on Render and its details can be found on the Render dashboard:

External-host: dpg-d6on5115pdvs73endql0-a.frankfurt-postgres.render.com
Port: 5432
Username: footprint_db_qe59_user
Database name: footprint_db_qe59
Password: 7GRaiRDfQnw2om7AxXPMFQ4dakk1uIPz

To connect from a local machine, install `psql` (included with [PostgreSQL](https://www.postgresql.org/download/)) and run:

```
psql -h dpg-d6on5115pdvs73endql0-a.frankfurt-postgres.render.com -U
footprint_db_qe59_user footprint_db_qe59
```

You will be prompted for the password.

Credentials

Account details for moderator:
Username: Bob

Data Management - Last Updated 25/03/26

Password: 4@$A3nM7_G1

Account details for test user:
Username: Ben
Password: 73%&Nma1!

Password requirements enforced at registration:
- Minimum 8 characters
- Username must be at least 3 characters, alphanumeric only

**These credentials work on both the deployed and staging websites*. All passwords are stored as BCrypt hashes in the database, plaintext passwords are never persisted.

User Permissions

The application uses two roles managed by Spring Security:
`USER`: Default role assigned to all newly registered accounts
`MODERATOR`: Elevated role with access to evidence review and challenge management

Access control rules

CSRF protection is enabled on all API endpoints.

Path	Access
`/`, `/login`, `/register`, static assets	Public (no login required)
`/api/moderator/**`	MODERATOR only
`/api/**`	Any authenticated user
All other pages	Any authenticated user

To promote a user to MODERATOR, connect to the database and run:

```
UPDATE users SET role='MODERATOR' WHERE username='your_username';
```

The user must log out and back in after a role change for the new permissions to take effect in their active session.